

UNIT 4

Positive Definite:

- If A is symmetric & +ve definite,

$$\rightarrow A = Q \Lambda Q^T \text{ [spectral decomposition]}$$

$$\rightarrow \text{SVD} \rightarrow A = Q \Sigma Q^T \text{ } (\Sigma \rightarrow \text{singular value matrix})$$

where $\Sigma = \Lambda$ [eigenvalues = singular values]

- If all singular values > 0 then matrix is invertible and $A^T A$ is positive definite.

Quadratic Forms:

$\rightarrow$ a real polynomial in variables $x_1, x_2, \dots, x_n$ such that every term in ~~quadratic~~ q has degree 2.

$$- q(x_1, x_2, \dots, x_n) = \sum_{i=1}^n c_i x_i^2 + \sum_{i < j} d_{ij} x_i x_j$$

where c_i & $d_{ij} \in \mathbb{R}$

q here is quadratic form.

- If $x_i x_j / d_{ij}$ all $= 0$ then q becomes diagonal.

$\rightarrow$ If $f(x, y) = a^2 + 2bxy + cy^2$ is +ve definite then,

$$- a > 0, c > 0$$

$$- ac > b^2$$

$\rightarrow f(x, y) = a^2 + 2bxy + cy^2$ is -ve definite when,

$$- a < 0$$

$$- ac > b^2$$

- When $ac = b^2$
 - if $a > 0$ then semi definite
 - if $a < 0$ then -ve semi definite.

- When $ac < b^2$
 - saddle points
 - indefinite.

- Matrix Form:

$$A \Rightarrow \begin{bmatrix} a & b \\ b & c \end{bmatrix}$$

$$f(x) \Rightarrow x_1, x_2, x_3 \Rightarrow$$

$$A = \begin{bmatrix} x_1 x_1 & x_1 x_2 & x_1 x_3 \\ x_2 x_1 & x_2 x_2 & x_2 x_3 \\ x_3 x_1 & x_3 x_2 & x_3 x_3 \end{bmatrix}$$

- Conditions for the Definiteness:

- $x^T A x > 0$ $\forall$ real nos $\neq 0$.
- $\lambda_i > 0$ $\forall$ eigen values
- All upper left submatrices of A must be the
- pivots without row exchange > 0 .
- R matrix exists with independent columns

$$A = R^T R$$

$$x^T A x = x^T R^T R x$$

$$= (R x)^T (R x)$$

$$= \|R x\|^2 > 0 \text{ [unless } x=0]$$

$$\bullet A = L D L^T \text{ [} A \text{ is symmetric / Hermitian matrix]} \\ = (L \sqrt{D}) (\sqrt{D} L^T)$$

let $R = \sqrt{D} L^T$ [Cholesky's decomposition]
 → splits pivots b/w L & L^T evenly.

And $A = Q \Lambda Q^T$ (Spectral Decomposition)

$$= (Q \sqrt{\Lambda}) (\cancel{Q} \sqrt{\Lambda} Q^T)$$

$$\Rightarrow R = \sqrt{\Lambda} Q^T$$

- For $A^T A$:

$$\rightarrow x^T (A^T A) x = (Ax)^T (Ax)$$

$$= \|Ax\|^2 \geq 0$$

- For $A+B$:

$\rightarrow$ If A & B are both definite then $A+B$ is also definite.

$$\rightarrow x^T (A+B) x = x^T A x + x^T B x > 0.$$

Semi Definite Matrices:

- $x^T A x \geq 0$ $\forall$ vectors x

- All eigenvalues of $A \geq 0$.

- Diagonal elements ≥ 0 .

- All pivots ≥ 0 .

- There is matrix R [dependent columns maybe] such that

$$\rightarrow A = R^T R$$

$$\rightarrow A = Q \Lambda Q^T$$

$$\Rightarrow x^T A x = x^T Q \Lambda Q^T x \\ = y^T \Lambda y$$

Singular Value Decomposition:

→ Matrix multiplied with vector results in scaling and rotation.

→ AA^T and $A^T A$:

- Both are symmetric matrices

- eigenvectors of $AA^T \rightarrow U_i$
eigenvectors of $A^T A \rightarrow V_i$ } singular vectors of A

- Both have same the eigenvalues & rank 'r' as A

→ Full SVD:

$$- \boxed{M = U \Sigma V^T}$$

where $U \Rightarrow m \times m$ (orthogonal)

$\Sigma \Rightarrow m \times n$ (diagonal)

$V \Rightarrow n \times n$ (orthogonal)

$M \Rightarrow m \times n$

→ Reduced / Thin SVD:

$$- \boxed{M = U_r \Sigma_r V_r^T}$$

where $r \Rightarrow$ rank of M

$U_r \Rightarrow m \times r$ (first r columns of U)

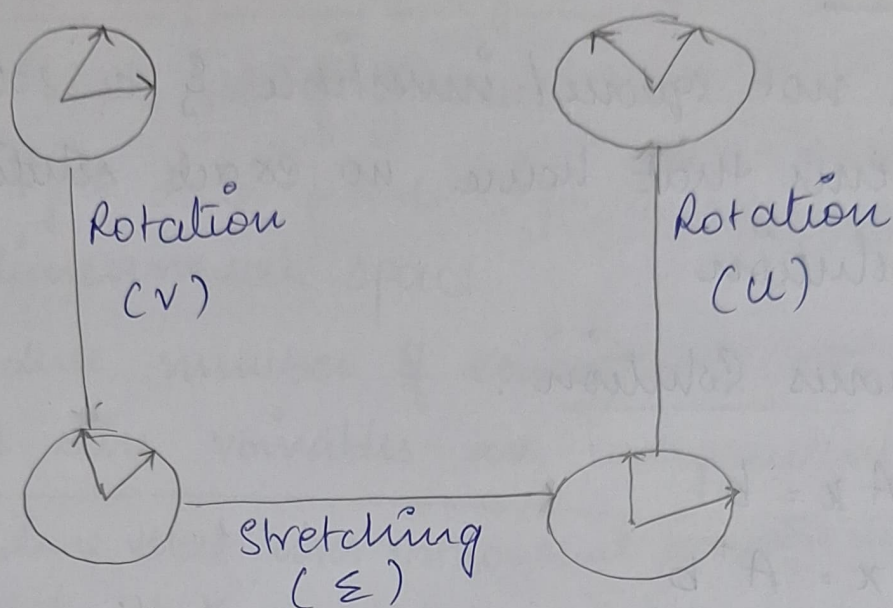
$V_r \Rightarrow n \times r$ (first r columns of V)

$\Sigma_r \Rightarrow r \times r$ (only non zero singular values)

$$U_i = \frac{A V_i}{\sigma_i} \text{ for } A^T A$$

$$V_i = \frac{U_i^T A}{\sigma_i} \text{ for } A A^T$$

where $\sigma_i = \sqrt{\lambda_i}$ → singular values are root of eigen values.



SVD: $\rightarrow \Sigma$ diagonal matrix of singular values in descending order.

$$\rightarrow \sigma_i = \sqrt{\lambda_i}$$

$\rightarrow$ columns of U are left singular vectors.

$\rightarrow$ columns of V are right singular vectors.

$\rightarrow$ For the definite matrices, Σ is Λ .

$$- U \Sigma V^T \text{ is } Q \Lambda Q^T$$

$\rightarrow$ For other symmetric matrices,

$-$ the eigenvalues in Λ become the Σ

$\rightarrow$ first r columns of $U = C(A)$

last $m-r$ columns of $U = N(A^T)$

first r columns of $V = C(A^T)$

last $n-r$ columns of $V = N(A)$

Pseudo inverse:

- when A is not square / invertible & to solve linear systems that have no exact solution / infinite solution.

- least squares solution:

$$\rightarrow Ax = b$$

$$\rightarrow x = A^+ b$$

$$- AA^+A = A$$

$$- (AA^+)^T = AA^+$$

$$- A^+AA^+ = A^+$$

$$- (A^+A)^T = A^+A$$

- SVD:

$$\rightarrow A^+ = V \Sigma^+ U^T$$

where $\Sigma^+ \rightarrow$ reciprocal of non zero singular values of Σ and transposing it

- How to find A^+ :

$$\rightarrow \text{find SVD } A = U \Sigma V^T$$

$\rightarrow$ find Σ^+ , add padding if needed

$\rightarrow$ compute

$$A^+ = V \Sigma^+ U^T$$

Principal Component Analysis: [PCA]

- gives ideal set of features.
- To project data from high dimensional space to low dimensional space.
- To reduce number of variables
- Make sure variables are independent

* Procedure that uses orthogonal transformation to convert set of correlated variables into set of linearly uncorrelated variables called principle components.

- Can be done in 2 ways → SVD
→ covariance matrix

- PCA o:

→ eigenvectors of covariance matrix C

$$C = \frac{1}{n} X X^T$$

- SVD:

→ columns of U are eigenvectors of C

→ squares of Σ values is eigenvalues of C

- Steps:

→ matrix X of $m \times n$

→ centre the data [subtract mean from row]

$$\tilde{X} = X - \text{mean}(X)$$

→ Find SVD

$$\tilde{X} = U \Sigma V^T$$

→ Project onto principal components

$$Z = U^T \tilde{X}$$

Image Processing:

- Grayscale Images:

→ 2D matrices

→ entries are pixel intensity

- Color Images:

→ 3 matrices of $m \times n$.

→ R, G, B

- Linear Transformation:

→ $x' = Ax$

→ matrix multiplication

- Image Blurring / Sharpening:

→ move filter across image, do multiplication addition with neighbouring pixels. [CONVOLUTION]

→ To blur use filter that averages nearby pixels

→ To sharpen use filter to emphasise edges.

- filter = small matrix

image = big matrix

Minimum Principles:

- Energy $\Rightarrow$ the definite quadratic functions.

$\Rightarrow$ derivative of quadratic is linear

$\Rightarrow$ Equate derivative to zero, point of minima.

- Minimum principle $\Rightarrow Ax = b$

Minimisation $\Rightarrow Ax = \lambda x$

- Minimise quad. forms $\Rightarrow x^T A x$

Optimising Through Calculus :

$$\Rightarrow f(x) = x^T A x - 2b^T x \quad \text{OR} \quad \frac{1}{2} x^T A x - b^T x$$

$$\Rightarrow \frac{d}{dx}(f) = 2Ax - 2b \quad [\text{gradient}]$$

$$\Rightarrow \text{make } \frac{d}{dx}(f(x)) = 0 \text{ for minimum} \Rightarrow Ax = b.$$

Least Squares :

$$E^2 = \|Ax - b\|^2$$

$$= (Ax - b)^T (Ax - b)$$

$$= (x^T A^T - b^T)(Ax - b)$$

$$= x^T A^T A x - \overbrace{x^T A^T b - b^T A x}^{2x^T A^T b} + b^T b \quad \because A \text{ is symmetric}$$

$$f(x) = x^T A^T A x - 2x^T A^T b + b^T b.$$

→ Take gradient & set to 0 :

$$\nabla f(x) = 2A^T A x - 2A^T b = 0$$

$$A^T A x = A^T b$$

$$\therefore x = (A^T A)^{-1} A^T b.$$

Back Propagation :

- Train neural networks by adjusting weight based on error using chain rule.

Working :

→ Forward Pass [goes through network to calculate loss and output]

→ Backward Pass [find gradient of loss with o/p (error) & send backward layer by layer]

calculate gradient with weight at every layer.

→ Update weight using gradient descent.

Stochastic Gradient Descent: [SGD]

- optimisation algorithm
- updates parameters using ~~loss~~-ve gradient to minimise loss function.
- step size ' η ' controls dist. moved in each iteration

Geometrical Interpretation:

- Gradients point to steepest ascent
- Negative gradients to steepest descent.
- SGD updates are vector subtractions
- loss surface is a multi variate function which needs to be minimised.

Example:

- Input - $x \in \mathbb{R}^n$
- Weights - $w \in \mathbb{R}^n$
- Bias - $b \in \mathbb{R}$
- Output - $y = w^T x + b$
- Activation - $\hat{y} = \sigma(y)$
- loss function - $L(\hat{y}, t) = \frac{1}{2} (\hat{y} - t)^2$ $t \rightarrow$ true label

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - t \quad \frac{\partial \hat{y}}{\partial y} = \sigma'(y)$$

For sigmoid:

$$\sigma(y) = \frac{1}{1 + e^{-y}} \quad \sigma'(y) = \sigma(y) * (1 - \sigma(y))$$

Gradient:

$$\frac{\partial L}{\partial w} = (\hat{y} - t) (\sigma'(y)) (x) \Rightarrow \text{with bias} \quad \frac{\partial L}{\partial b} = (\hat{y} - t) * \sigma'(y)$$

→ SGD update :

$$w_{\text{new}} = w - \eta * dL/dw$$

$$b_{\text{new}} = b - \eta * \frac{dL}{db}$$